

cmclab4

September 17, 2026

0.1 Lab 4

0.1.1 Charlie Caravati

```
[68]: import numpy as np
```

3.1.1

$$g(x) = x - \frac{f(x)}{f'(x)}$$

We need $g'(x) < 1$ for the basin of convergence.

$$g'(x) = 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{f'(x)^2} < 1$$

$$g'(x) = 1 - \frac{f'(x)^2 - f(x)f''(x)}{f'(x)^2} = 1 - \frac{f'(x)^2}{f'(x)^2} - \frac{f(x)f''(x)}{f'(x)^2}$$

$$\frac{f(x)f''(x)}{f'(x)^2} < 1$$

```
[34]: f = lambda x: x**3+x-4
      fp = lambda x: 3*x**2 + 1
      f2p = lambda x: 6*x
```

3.1.2

```
[48]: def bisection(f,fp, f2p, a,b, nmax):

      # first verify there is a root we can find in the interval
      fa = f(a)
      fb = f(b)
      if (fa*fb>0):
          ier = 1
          astar = a
          return [astar, ier]

      # verify end points are not a root
      if (fa == 0):
          astar = a
          ier =0
          return [astar, ier]

      if (fb ==0):
```

```

    astar = b
    ier = 0
    return [astar, ier]

d = 0.5*(a+b)
fd = f(d)
fpd = fp(d)
f2pd = f2p(d)

count = 1

while (((fd * f2pd)/(fpd**2)) > 1) and count < nmax:
    count += 1
    d = 0.5*(a+b)
    fd = f(d)
    fpd = fp(d)
    f2pd = f2p(d)
    if (fd == 0):
        astar = d
        ier = 0
        return [astar, ier]
    if (fa*fd<0):
        b = d
    else:
        a = d
        fa = fd
    astar = d
    print('count = ', count)
    print('midpoint in basin:', astar)
    return [astar]

```

[49]: basin = bisection(f, fp, f2p, 1, 17, 10000)

```

count = 1
midpoint in basin: 9.0

```

3.1.3

Yes, we need to input first and second derivatives as well to identify the basin of convergence.

3.1.4

```

[56]: def bisec_newt(f,fp, f2p, a,b, nmax, tol):

    # first verify there is a root we can find in the interval
    fa = f(a)
    fb = f(b)
    if (fa*fb>0):
        ier = 1

```

```

    astar = a
    return [astar, ier]

# verify end points are not a root
if (fa == 0):
    astar = a
    ier = 0
    return [astar, ier]

if (fb == 0):
    astar = b
    ier = 0
    return [astar, ier]

d = 0.5*(a+b)
fd = f(d)
fpd = fp(d)
f2pd = f2p(d)

count = 0

while (((fd * f2pd)/(fpd**2)) > 1) and count < nmax:
    count += 1
    d = 0.5*(a+b)
    fd = f(d)
    fpd = fp(d)
    f2pd = f2p(d)
    if (fd == 0):
        astar = d
        ier = 0
        return [astar, ier]
    if (fa*fd < 0):
        b = d
    else:
        a = d
        fa = fd
p0 = d
ier = 0

# Newton Method
for i in range(nmax):
    p1 = p0 - f(p0)/fp(p0)
    if (abs(p1-p0) < tol):
        pstar = p1
        return pstar
    p0 = p1
pstar = p1

```

```
return pstar
```

3.1.5

This function guarantees convergence if bisection conditions are satisfied but might not be the most efficient method and requires the second derivative of the function.

3.1.6

```
[50]: f = lambda x: np.exp(x**2+7*x-30) - 1  
      fp = lambda x: (2*x+7)*np.exp(x**2+7*x-30)  
      f2p = lambda x: (2+(2*x+7)**2)*np.exp(x**2+7*x-30)
```

```
[67]: root = float(bisec_newt(f, fp, f2p, 2, 4.5, 10000, .000001))  
  
      print(f"The root is {root}.")
```

The root is 3.0.